

Beyond Arabic Digits: Evaluating Sudoku Solving Under Symbol-Set Variation

Arnav Bharti

September 2026

Abstract

This study evaluates whether a large language model preserves Sudoku-solving ability when the nine values are renamed by bijective symbol systems. Sudoku provides exact, deterministic scoring: a returned grid must preserve every clue and satisfy every row, column, and 3×3 box. A certified dataset contains 300 unique-solution puzzles divided equally among easy, medium, and hard tiers. The selected model, GPT-OSS-120B, uses hidden Harmony analysis while only its final channel is scored. It solved all five qualification puzzles. On the frozen 540-request main benchmark, it solved 372/540 requests (68.9%) with no operational failures. Accuracy ranged from 61.7% for Greek letters to 75.0% for Arabic digits and uppercase Latin, while difficulty accuracy declined from 95.0% on easy puzzles to 67.2% on medium and 44.4% on hard puzzles. All 75 basic symbol controls in the input/output-cross experiment were correct, but its four Sudoku conditions scored between 10/15 and 12/15. Token-length accuracy was non-monotonic at 9/15, 13/15, and 10/15. In the binding experiment, ordinary digits, ordinary number words, and neutral nonce labels each scored 12/15, while permuted digits and conflicting number words each scored 10/15; six mappings over the same uppercase alphabet ranged from 10/15 to 13/15. Exploratory prompt and format ablations produced no uniformly beneficial prompt or output format. These results demonstrate representation sensitivity but do not reduce it to Unicode handling, transformed-output generation, token length, or one prompt-format choice.

1 Introduction

Large language models are increasingly evaluated on tasks described as reasoning benchmarks, yet observed performance may depend on the surface representation used to express an otherwise unchanged problem. This dependence matters because logical validity should not change when the values of a constraint-satisfaction problem are consistently renamed. A system that has learned the underlying constraints should, in principle, transfer across bijective symbol systems; a system dependent on familiar tokens, learned semantic associations, or fragile output routines may not.

Sudoku is useful for studying this issue because correctness is exact rather than subjective. A candidate solution can be checked cell by cell against the original clues and against the row, column, and box constraints. The same abstract puzzle can also be encoded under multiple alphabets without changing its solution structure. This produces a controlled test of representation invariance in which the logical problem is fixed and only its visible labels change.

The research question is: when an LLM can solve a Sudoku puzzle, does it preserve that ability when Arabic numerals are replaced by another bijective symbol system? The broader experimental programme separates possible failures in input decoding, symbol-to-value binding, constraint reasoning, output encoding, tokenization, Unicode handling, and formatting. The work reported here establishes the dataset, evaluator, model qualification, balanced pilot, complete main representation benchmark, paired retention analysis, and the input/output, token-length, binding, and prompt-format mechanism experiments for GPT-OSS-120B.

2 Background

2.1 Sudoku as an exactly verifiable constraint problem

Let $G = \{g_{r,c} \mid r, c \in \{1, \dots, 9\}\}$ denote the 81 grid cells and let $D = \{1, \dots, 9\}$ be the value domain. A completed grid is correct only if it preserves every clue and, for every row r , column c , and box b ,

$$\{g_{r,1}, \dots, g_{r,9}\} = D, \quad (1)$$

$$\{g_{1,c}, \dots, g_{9,c}\} = D, \quad (2)$$

$$\{g_{r,c} : (r, c) \in b\} = D. \quad (3)$$

This definition permits deterministic evaluation of all cells and constraints. Puzzle difficulty is not determined by clue count alone; technique complexity and dependencies among deductions also contribute [4]. Existing Sudoku evaluations similarly report lower full-grid solve rates as task complexity increases [6], while solver-trace training demonstrates that language models can learn search and constraint propagation on logic puzzles [7].

2.2 Representation invariance

Define abstract values $V = \{V_1, \dots, V_9\}$ and a visible alphabet $S = \{s_1, \dots, s_9\}$. A representation is a bijection $\sigma : V \leftrightarrow S$. If p is a Sudoku puzzle and f is the model-and-parser pipeline, exact invariance requires

$$\sigma^{-1}(f(\sigma(p))) = f(p) \quad (4)$$

whenever both sides return completed grids. Consistency alone is insufficient because a model can reproduce the same invalid answer under multiple encodings; Sudoku adds an independent validity test. Related controlled transformations have revealed sensitivity in mathematical reasoning benchmarks [2].

2.3 Binding, tokenization, and verification

Arbitrary mappings are learnable: symbol tuning can improve adherence to label remappings [9]. Nevertheless, consistently producing an 81-cell grid requires sustained binding across many positions. Digit-renaming experiments in elementary arithmetic likewise show failures under bijective changes [10], but such failures can mix tokenization, memorized associations, mapping, and generation effects.

Tokenization provides another possible source of variation. Vocabulary construction affects model performance [1, 5], and visible symbols can differ in token count, byte length, and Unicode structure. Finally, verbal self-assessment is not exact verification: a model’s assertion that its grid is valid does not supersede deterministic checking [8].

3 Methodology

3.1 Study logic

The study asks whether Sudoku solving is invariant to a change in visible value labels. The abstract values are $V = \{V_1, \dots, V_9\}$. A representation is a bijection from these values to nine visible symbols. Applying a bijection changes how the puzzle looks, but it does not change any clue position, row, column, box, or solution. Therefore, a difference in accuracy between two representations cannot be caused by a different logical puzzle.

The experiments are ordered around a possible failure chain:

$$\text{decode the input} \longrightarrow \text{bind labels to values and solve} \longrightarrow \text{encode the output.} \quad (5)$$

The main benchmark first measures the total representation effect. Later experiments then change one part of the pipeline at a time. They test input and output separately, token length, arbitrary mappings,

semantic conflict, prompt and format choices, and answer revision. This order is important: the main benchmark can show that a representation effect exists, while the mechanism experiments are needed to explain where it comes from.

3.2 Shared experimental controls

3.2.1 Model and inference

The reported experiments use the open-weight GPT-OSS-120B checkpoint [3]. Its repository identifier is `openai/gpt-oss-120b`, and the pinned revision is `b5c939de8f754692c1647ca79fbf85e8c1e70f8a`. Inference runs through vLLM on one NVIDIA H100 GPU with 12 CPU cores and 96 GB of host memory. The chat template requests high reasoning effort. Sampling uses temperature 1 and top- p 1, with master seed 20260826. The model context and maximum generation length are 131,072 tokens.

GPT-OSS returns Harmony analysis and final channels. The backend stores them separately, and only the final channel is evaluated. The visible reasoning trace is not an outcome of this thesis. Scheduler waiting time is also excluded from latency; latency measures model generation for one request.

3.2.2 Open-weight model screening and selection

The final benchmark model was selected through staged local screening rather than assumed in advance. Table 1 separates completed inference evidence from models that were only prepared for execution. This distinction is necessary because installation, configuration, or a pending scheduler job does not constitute a model result.

Table 1: Open-weight models evaluated or prepared during model selection. “Prepared” means that a reproducible local profile or model installation existed, but no completed scored screen supports an accuracy claim.

Model	Execution status	Basis for continuing or stopping
NVIDIA Llama-3.3 Nemotron Super 49B v1.5 FP8	Evaluated in four diagnostic settings	Reasoning-enabled generation produced long, contradictory output rather than a clean final grid. Three answer-only variants—greedy, sampled, and regex-constrained—scored 0/5, 0/15, and 0/15, respectively. The constraint fixed formatting but not clue preservation or Sudoku validity, so the model was not advanced.
Qwen3.8-27B (earlier non-FP8 screen)	Evaluated on one puzzle per difficulty	It solved the selected easy and hard puzzles, but the medium run did not return a final grid. The mixed three-item diagnostic did not justify committing the full study to this checkpoint once a stronger hidden-reasoning candidate was available.
GPT-OSS-120B	Evaluated and selected	It solved all three timing puzzles and all five qualification puzzles while keeping Harmony analysis separate from the scored final channel. It then completed the balanced pilot with 47/60 correct and no operational failures, satisfying both capability and output-separation requirements.
Qwen3.8-27B-FP8	Prepared for replication; timing jobs queued	The official FP8 checkpoint was pinned and verified locally on Sharanga. One-puzzle timing jobs for easy, medium, and hard tiers were submitted, but no completed scored result was available for this draft revision.
Qwen3.5-122B-A10B FP8	Prepared; no completed scored screen	The checkpoint was installed and a diagnostic job was queued, but that job was cancelled while pending when the study was narrowed to completing GPT-OSS first. No Sudoku-accuracy conclusion is drawn for Qwen3.5.
Mistral Small 4 119B and Mistral Medium 3.5 128B	Prepared or configured; no completed scored screen	These dense two-H200 candidates were investigated as alternatives, but neither supplied completed screening evidence before the experiment was narrowed to one fully qualified model. They were omitted for scope and compute control, not because of a measured Sudoku failure.

Other open-weight families, including GLM and Kimi, were discussed during candidate search but were not run as scored experiments and therefore are not reported as evaluated models. The resulting thesis is a controlled single-model representation study, not a comparative model leaderboard. The discarded screens informed model selection only; they did not alter the frozen puzzle samples or the confirmatory request set.

3.2.3 Representations

The main benchmark uses nine ordered alphabets. Each item in an alphabet represents the abstract value in the same position:

Family	Ordered labels for V_1 through V_9
Arabic digits	1, 2, 3, 4, 5, 6, 7, 8, 9
Devanagari numerals	१, २, ३, ४, ५, ६, ७, ८, ९
Bengali numerals	১, ২, ৩, ৪, ৫, ৬, ৭, ৮, ৯
Uppercase Latin	A, B, C, D, E, F, G, H, I
Lowercase Latin	a, b, c, d, e, f, g, h, i
Greek letters	$\alpha, \beta, \gamma, \delta, \epsilon, \zeta, \eta, \theta, \iota$
Abstract symbols	triangle, square, circle, star, multiplication sign, dagger, double dagger, section sign, paragraph sign
Emoji	red, orange, yellow, green, blue, purple, brown, black, and white circles
Nonce labels	KAV, MIP, ZOT, RUL, BEK, DAX, PEV, NUG, WIF

All symbols are stored in Unicode NFC form. The encoder writes UTF-8 text, and a round-trip test requires decoding an encoded grid to recover the original abstract grid exactly. The dot is the default empty-cell marker and cannot be a value symbol.

3.2.4 Standard prompt and response

The standard prompt names the valid alphabet, shows the transformed puzzle, and states four requirements: every row, column, and 3×3 box must contain each value exactly once, and the given cells must not change. It also states that visible symbols are labels and that one mapping must be used consistently. The requested answer is exactly nine lines of nine single-space-separated output symbols, with no explanation. Apart from the declared alphabet and transformed cell values, the puzzle, wording, output format, model settings, and evaluator remain fixed.

The exact parser does not search prose for a plausible grid. It requires nine rows, 81 cells, one space between adjacent symbols, and membership in the requested alphabet. A candidate is correct only if it is parseable, preserves every clue, equals the certified solution, and satisfies all nine rows, nine columns, and nine boxes. The evaluator can attach several error labels to one answer: changed clue, row, column or box violation, invalid symbol, wrong row or cell count, format error, Unicode error, refusal, missing final answer, or truncation. If one consistent permutation changes the returned grid into the certified solution, it is a global binding error. Other inconsistent cell mappings are local mapping errors.

An API error or timeout produces `NOT_EVALUATED`. It is reported as an operational failure and is not counted as an incorrect Sudoku answer. An answer that finishes normally but is invalid, malformed, missing, refused, or truncated is an evaluated model failure. Backend failures follow the fixed retry policy of at most five attempts with exponential delays. The raw status, token counts, finish reason, timestamps, and generation latency are saved with every result.

3.3 Experiment 1: dataset and evaluator validation

Question. Can later failures be attributed to the model rather than to an ambiguous puzzle, a wrong reference solution, or a Unicode and parser defect?

Method. A seeded exact solver first generates a completed grid. Clues are then considered for removal in seeded random order. A removal is kept only when an exact solution counter, which stops after two solutions, finds exactly one solution. The generator targets 40–46 clues for easy candidates, 24–27 for medium candidates, and 22–26 for hard candidates. Candidates whose measured difficulty does not match the requested tier are rejected, as are duplicate puzzle hashes.

Difficulty is assigned by a deterministic technique solver. A puzzle is easy if naked and hidden singles solve it. It is medium if solving also needs an implemented intermediate technique: naked pair, hidden pair, pointing pair, or box-line reduction. If these techniques stop before completion and exact search is required, the puzzle is hard. The certificate records deduction steps, techniques used, whether backtracking was required, maximum search depth, and search nodes.

The final dataset contains 300 unique puzzles: 100 easy, 100 medium, and 100 hard. Each record stores its ID, compact puzzle and solution, clue mask and count, generation seed and version, SHA-256 puzzle hash, difficulty certificate, and solver certificate. A separate audit rechecks record structure, hashes, uniqueness, exact solution count, stored solution, difficulty, and certificate. File hashes are also stored in a manifest. This experiment therefore establishes both valid test items and a deterministic scoring pipeline before any model comparison.

3.4 Experiment 2: model screen, qualification, and pilot

Question. Can the selected model and compute setup complete exact Sudoku grids across difficulty levels and transformed alphabets, and are the selected puzzles informative rather than all too easy or all too hard?

Method. A timing screen uses one reproducibly selected puzzle from each difficulty. Its purpose is to measure natural reasoning time and to set scheduler wall time without limiting reasoning to a small token budget. Qualification then uses five separate easy puzzles. They are assigned once each to Arabic digits, Greek letters, emoji, nonce labels, and Devanagari numerals. The model must solve all five with no operational failure before the protocol can be frozen.

The pilot uses 15 different puzzles, with five from each difficulty tier. Every puzzle is shown under Arabic digits, uppercase Latin, Greek, and emoji, giving $15 \times 4 = 60$ requests. The same puzzle set in every representation makes the representation comparison balanced. Pilot accuracy is inspected by difficulty and representation, together with latency, output length, truncation, and operational failures. Pilot puzzles are excluded from the main sample. Thus, any choice informed by pilot performance cannot change the confirmatory outcomes for those same items.

3.5 Experiment 3: representation registry and protocol freeze

Question. Can a representation effect be related to measurable symbol cost, and can the confirmatory design be protected from changes made after seeing results?

Method. For every main-alphabet symbol, the registry records its Unicode code points, UTF-8 byte length, code-point count, grapheme count, token count and token IDs in isolation, token count after a leading space, and the token cost of the full alphabet row. For every puzzle and alphabet it also records clue count, total tokens used by visible clues, mean clue-token cost, and total prompt tokens. Exact token IDs are available because the selected model has a local tokenizer.

After qualification and the pilot, the code freezes the dataset hash, master seed, sample IDs, model revision, inference and retry settings, experiment shard counts, and a digest of all numbered scripts and library source files. It also freezes the exact request-ID set for each experiment. Later scripts refuse to run if the configuration, sample plan, or source digest differs from the frozen protocol.

Sample selection is deterministic and stratified. The 15-puzzle pilot is disjoint from the main sample. The main sample contains 60 puzzles, with 20 per difficulty. A 15-puzzle mechanism sample, with five per difficulty, is selected from the main sample. A nine-puzzle ablation sample, with three per difficulty, is selected from the mechanism sample. This nesting allows later experiments to reuse the exact main-benchmark answers instead of generating a different baseline.

3.6 Experiment 4: main representation benchmark

Question. Does changing only the visible value labels change complete-grid accuracy?

Method. Every one of the 60 main puzzles is presented under all nine alphabets. This gives $60 \times 9 = 540$ requests for the selected model. All logical content and inference settings are held fixed. The primary summary for each family is the number correct divided by the number evaluated, with operational failures shown separately and Wilson 95% confidence intervals for proportions. Results are also divided by difficulty, and diagnostic failure labels are counted.

For reliable long runs, request IDs are SHA-256 hashes of canonical request data. A second hash assigns the 540 requests to six deterministic shards. Results are appended to JSONL after each request. A resumed run skips existing request IDs, and a shard is complete only when all eligible requests are present and a completion marker has been written. These controls prevent a scheduler timeout from losing finished work or causing duplicate inference.

3.7 Experiment 5: paired Arabic retention analysis

Question. Among puzzles the model can solve with ordinary digits, how often does it retain that ability after the values are renamed?

Method. This experiment makes no new model calls. For each non-Arabic family s , let B be the main-sample puzzles answered correctly in Arabic. The retention and representation-failure rates are

$$R_s = \frac{|\{p \in B : p \text{ is correct under } s\}|}{|B|}, \quad F_s = 1 - R_s. \quad (6)$$

They are reported overall and by difficulty. This conditional analysis directly answers the thesis question, because it asks whether a demonstrated Arabic solution survives the transformation. All paired Arabic-only and transformed-only successes are also compared with a two-sided exact McNemar test. Cell, constraint, binding, termination, and operational failures remain separate.

3.8 Experiment 6: input/output cross and basic symbol controls

Question. If transformed performance is lower, is the main problem reading the transformed puzzle, solving while maintaining the mapping, or producing a long transformed answer?

Method. The 15 mechanism puzzles are tested in four conditions: Arabic input to Arabic output (A), Greek input to Greek output (B), Greek input to Arabic output (C), and Arabic input to Greek output (D). Cross-alphabet prompts give the complete symbol mapping. Conditions A and B reuse the matching main-benchmark results; C and D are new calls. If C is close to A while B is lower, Greek output generation is implicated. If C also remains below A, input decoding or symbol binding is still a likely source. Comparing D with A gives a more direct test of Greek output cost when input decoding is unchanged.

Five no-Sudoku controls are added for every mechanism puzzle: copy a Greek row exactly, translate a short Greek sequence to digits, retrieve one coordinate from a completed Greek grid, count one symbol in a completed grid, and convert a supplied completed Greek grid to Arabic. They use exact-text scoring. These controls show whether a failure can already be produced by copying, lookup, counting, or conversion without Sudoku search. In total the experiment contains 135 recorded conditions; 30 Sudoku baselines are reused and 105 requests require new inference.

3.9 Experiment 7: token-length mechanism

Question. Does accuracy decrease because some visible labels require more model tokens, rather than because their abstract meaning is different?

Method. Using the selected model’s tokenizer and the master seed, the code searches neutral upper-case nonce strings. It constructs three nine-label alphabets whose labels require exactly one, two, or three tokens both in isolation and after a leading space. The 15 mechanism puzzles are solved under all three alphabets, giving 45 requests. If any token-length group cannot supply nine verified distinct labels, the experiment stops instead of silently changing its design.

The analysis relates correctness to total prompt tokens, mean clue tokens, mean UTF-8 bytes, and mean code points using logistic regression with standardized predictors. Token length is supported as a mechanism only if accuracy changes in the controlled alphabets. A remaining family effect would show that token count is incomplete, not that tokenization is irrelevant.

3.10 Experiment 8: arbitrary binding and semantic conflict

Question. Is performance sensitive to an arbitrary mapping itself, and do familiar labels interfere when they are assigned meanings that conflict with their usual meanings?

Method. The 15 mechanism puzzles use the standard uppercase mapping and five deterministic random permutations of the same letters. Variation across these five mappings measures sensitivity to the particular binding rather than to the alphabet alone. The same puzzles also use ordinary digits, a

fixed permutation of digits, ordinary English number words, a fixed conflicting permutation of number words, and neutral nonce labels. A larger penalty for conflicting familiar labels than for neutral labels supports semantic interference.

There are 11 conditions per puzzle, or 165 recorded conditions. Standard uppercase, ordinary digits, and nonce labels reuse matching main-benchmark results. The remaining eight conditions per puzzle require 120 new calls. Seed-fixed permutations ensure that the comparison can be reproduced exactly.

3.11 Experiment 9: prompt and format ablations

Question. Could the measured representation effect be reduced or explained by more explicit instructions, a declared mapping, a different output format, a different empty marker, or case and spacing conventions?

Method. This is an exploratory one-factor-at-a-time study on the nine-puzzle ablation sample. Greek puzzles are tested with four rule styles: minimal, explicit constraints, constraints plus alphabet, and the fully explicit standard prompt. Three mapping styles provide only the alphabet, a mapping to digits, or a mapping to abstract values. Four output formats require spaced rows, compact rows, one 81-symbol string, or a JSON array. Four empty markers are compared: dot, zero, underscore, and the word EMPTY. Finally, uppercase and lowercase forms of Latin and nonce labels are compared.

These 19 variants give $9 \times 19 = 171$ requests. Each output format has its own exact normalizer before the common Sudoku evaluator. Ablation results do not retroactively change the frozen main protocol.

3.12 Experiment 10: self-revision and checker-guided revision

Question. Can the model detect and repair an incorrect grid, and is exact checker feedback more useful than an unaided request to check again?

Method. The nine ablation puzzles are used in Arabic, Greek, and emoji form. For each puzzle-representation pair, four independent branches start from the same saved main-benchmark answer: no revision, one self-revision, two self-revisions, and one checker-guided revision. Reusing the same initial answer removes variation caused by generating four different baselines. Each branch has its own conversation after that initial answer.

The self-revision prompt asks the model to check cell count, clues, rows, columns, and boxes. In the checker-guided branch, the deterministic evaluator returns error descriptions, such as a changed clue or a duplicate and missing symbol, but never supplies the correct cell value. A correct initial answer receives no unnecessary checker-guided call. Every intermediate and final grid is saved and scored. The analysis reports initial and final accuracy, wrong answers fixed, correct answers made wrong, model calls, token use, and total generation time. The 108 branch records require between 81 and 108 new calls, depending on how many checker-guided baselines are already correct.

3.13 Analysis and reporting

The unit of correctness is the complete final grid; partial credit and visible reasoning quality are not used. Accuracy summaries exclude only NOT_EVALUATED operational cases from their denominator and always report those cases separately. Counts accompany percentages so that small groups remain visible. Wilson 95% intervals are used for accuracy proportions, exact McNemar tests are used for paired main outcomes, and the token mechanism uses the regression described above. Results are reported by representation and difficulty together with format compliance, clue preservation, row/column/box validity, local and global mapping errors, truncation, latency, token counts, and operational status. This reporting scheme prevents a correctly formatted but invalid grid, or a transport failure, from being described as successful reasoning.

4 Results

4.1 Qualification and timing

GPT-OSS-120B solved all five qualification requests and produced no operational failures. Mean qualification latency was 44.6 s per request. In the one-puzzle timing screen, the easy, medium, and hard puzzles were all solved correctly in 36.1 s, 254.6 s, and 227.4 s, respectively. These measurements motivated wall-time-based execution rather than a small reasoning-token budget.

4.2 Balanced pilot

The model solved 47/60 pilot requests (78.3%). All 60 requests completed at the inference layer and none had an operational error. Mean generation latency was 252.5 seconds, the median was 194.9 seconds, and mean completion length was 46944 tokens. Two responses reached the generation ceiling and did not contain a complete final grid.

Table 2: Pilot accuracy by difficulty. Intervals are Wilson 95% confidence intervals.

Difficulty	Correct	Accuracy (%)	95% CI (%)
Easy	20/20	100.0	[83.9, 100.0]
Hard	14/20	70.0	[48.1, 85.5]
Medium	13/20	65.0	[43.3, 81.9]

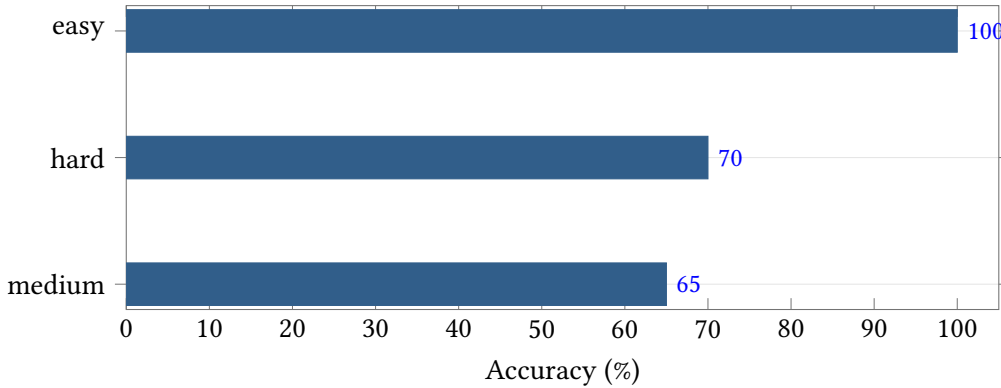


Figure 1: Pilot accuracy by difficulty ($n = 20$ per tier).

Table 3: Pilot accuracy by representation.

Representation	Correct	Accuracy (%)	95% CI (%)
Arabic digits	14/15	93.3	[70.2, 98.8]
Emoji	13/15	86.7	[62.1, 96.3]
Greek letters	10/15	66.7	[41.7, 84.8]
Uppercase Latin	10/15	66.7	[41.7, 84.8]

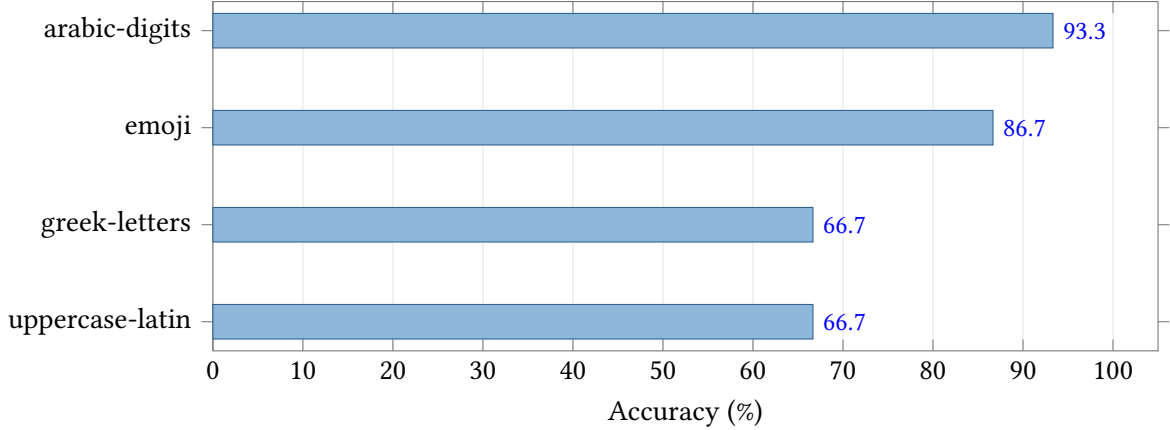


Figure 2: Pilot accuracy by representation ($n = 15$ per representation).

Arabic digits had the highest pilot accuracy at 14/15, followed by emoji at 13/15. Greek and uppercase Latin each yielded 10/15. Because every representation used the same balanced set of five puzzles per tier, these differences are not caused by a different difficulty composition. Difficulty accuracy was 20/20 for easy, 13/20 for medium, and 14/20 for hard. The medium-hard ordering in this 20-item-per-tier sample was not monotonic.

4.3 Main representation benchmark

All six deterministic shards completed, giving 540/540 recorded requests and valid completion markers. GPT-OSS-120B solved 372/540 requests (68.9%) with 0 operational failures. Mean and median generation latency were 290.8 and 240.5 seconds, and mean completion length was 54756 tokens. Twenty-four responses reached the generation ceiling. Exact row formatting was preserved in 515/540 responses. Of the 472 responses that could be decoded as grids over the requested alphabet, 458 preserved every given cell.

Table 4: Main-benchmark accuracy by difficulty. Each tier contains 20 puzzles under nine representations.

Difficulty	Correct	Accuracy (%)	95% CI (%)
Easy	171/180	95.0	[90.8, 97.3]
Hard	80/180	44.4	[37.4, 51.7]
Medium	121/180	67.2	[60.1, 73.7]

Table 5: Main-benchmark accuracy by representation. Each representation uses the same 60 puzzles.

Representation	Correct	Accuracy (%)	95% CI (%)
Abstract symbols	38/60	63.3	[50.7, 74.4]
Arabic digits	45/60	75.0	[62.8, 84.2]
Bengali numerals	43/60	71.7	[59.2, 81.5]
Devanagari numerals	39/60	65.0	[52.4, 75.8]
Emoji	38/60	63.3	[50.7, 74.4]
Greek letters	37/60	61.7	[49.0, 72.9]
Lowercase Latin	44/60	73.3	[61.0, 82.9]
Nonce labels	43/60	71.7	[59.2, 81.5]
Uppercase Latin	45/60	75.0	[62.8, 84.2]

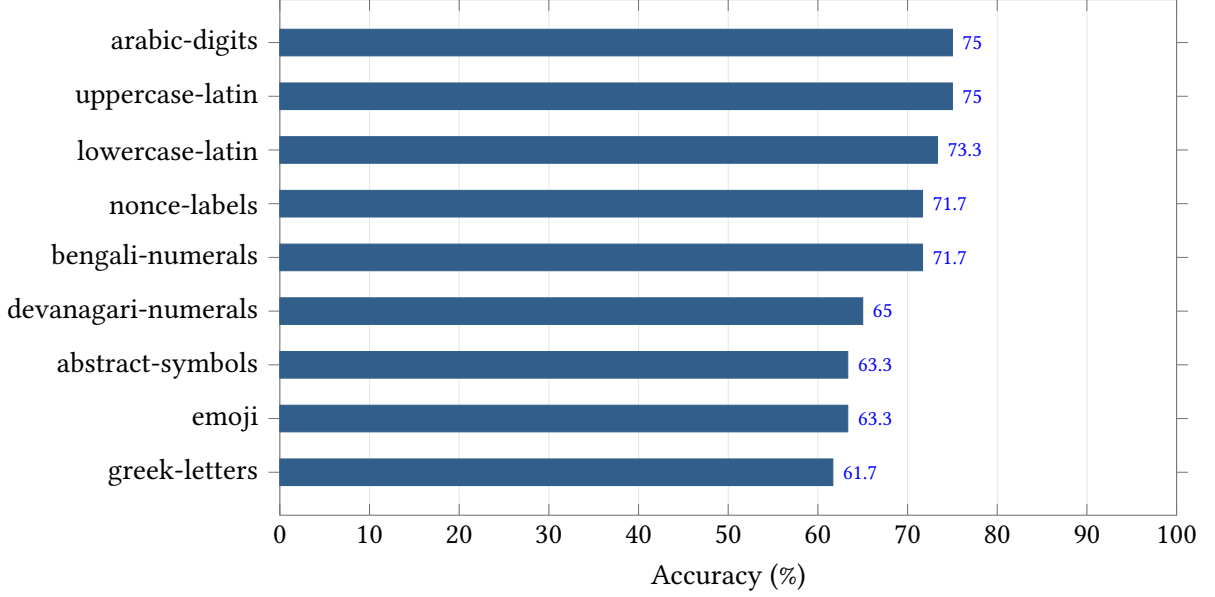


Figure 3: Main-benchmark accuracy by representation ($n = 60$ each).

Accuracy declined monotonically with measured difficulty: 171/180 (95.0%) on easy, 121/180 (67.2%) on medium, and 80/180 (44.4%) on hard requests. Representation accuracy ranged from 37/60 (61.7%) for Greek letters to 45/60 (75.0%) for both Arabic digits and uppercase Latin. The equal Arabic and uppercase-Latin marginal accuracy does not imply identical item-level behaviour.

4.4 Paired Arabic retention

Arabic digits were correct for 45 of the 60 main puzzles. Table 6 conditions each transformed representation on those 45 demonstrated Arabic successes. Retention ranged from 31/45 (68.9%) for abstract symbols to 37/45 (82.2%) for lowercase Latin. The discordant columns report Arabic-only/transformed-only successes across all 60 paired puzzles.

Table 6: Retention among the 45 puzzles solved correctly with Arabic digits. McNemar p values are exact and two-sided.

Representation	Retained	Retention (%)	Discordant	p
Abstract symbols	31/45	68.9	14/7	0.189
Bengali numerals	36/45	80.0	9/7	0.804
Devanagari numerals	36/45	80.0	9/3	0.146
Emoji	32/45	71.1	13/6	0.167
Greek letters	36/45	80.0	9/1	0.021
Lowercase Latin	37/45	82.2	8/7	1.000
Nonce labels	36/45	80.0	9/7	0.804
Uppercase Latin	36/45	80.0	9/9	1.000

4.5 Input/output cross and symbol controls

The input/output-cross experiment completed all 135 recorded conditions with 0 operational failures. Of these, 30 baseline Sudoku results were reused from the main benchmark and 105 were new inference calls. Table 7 reports every condition.

Table 7: Input/output-cross and no-Sudoku control accuracy ($n = 15$ per condition).

Condition	Correct	Accuracy (%)	95% CI (%)
A: Arabic input, Arabic output	12/15	80.0	[54.8, 93.0]
B: Greek input, Greek output	12/15	80.0	[54.8, 93.0]
C: Greek input, Arabic output	10/15	66.7	[41.7, 84.8]
D: Arabic input, Greek output	11/15	73.3	[48.0, 89.1]
Coordinate retrieval	15/15	100.0	[79.6, 100.0]
Greek row copy	15/15	100.0	[79.6, 100.0]
Grid conversion	15/15	100.0	[79.6, 100.0]
Mapping translation	15/15	100.0	[79.6, 100.0]
Occurrence count	15/15	100.0	[79.6, 100.0]

All five basic symbol controls were correct for all 15 puzzles: copying, short mapping translation, coordinate retrieval, occurrence counting, and completed-grid conversion each scored 15/15. The four Sudoku conditions were lower: Arabic input to Arabic output and Greek input to Greek output each scored 12/15, Greek input to Arabic output scored 10/15, and Arabic input to Greek output scored 11/15. By difficulty across the four Sudoku conditions, accuracy was 19/20 on easy, 18/20 on medium, and 8/20 on hard puzzles. The two truncated responses occurred on hard Sudoku requests.

4.6 Token-length mechanism

The token-length experiment completed all 45 requests with 0 operational failures and 5 truncated responses. Accuracy was not monotonic in the controlled label length.

Table 8: Accuracy under neutral alphabets requiring one, two, or three tokens per label ($n = 15$ each).

Condition	Correct	Accuracy (%)	95% CI (%)
One-token labels	9/15	60.0	[35.7, 80.2]
Two-token labels	13/15	86.7	[62.1, 96.3]
Three-token labels	10/15	66.7	[41.7, 84.8]

The one-token labels scored 9/15 (60.0%), the two-token labels scored 13/15 (86.7%), and the three-token labels scored 10/15 (66.7%). By difficulty across the three alphabets, accuracy was 15/15 on easy, 13/15 on medium, and 4/15 on hard puzzles. The five truncations comprised four hard requests and one medium request. The preregistered multivariable logistic fit could not be estimated because its information matrix was singular: prompt length, mean clue-token cost, byte length, and code-point length co-varied deterministically in this 45-request construction. Accordingly, the descriptive result rejects a simple monotonic token-count account but does not identify an independent coefficient for token length.

4.7 Arbitrary binding and semantic conflict

The binding experiment completed all 165 records with 0 request-level operational failures. It solved 124/165 conditions (75.2%) and produced 7 truncated responses. Table 9 reports all mappings.

Table 9: Binding accuracy and truncation by condition ($n = 15$ each).

Condition	Correct	Accuracy (%)	Truncated
Standard uppercase	11/15	73.3	1
Uppercase permutation 1	11/15	73.3	1
Uppercase permutation 2	11/15	73.3	1
Uppercase permutation 3	13/15	86.7	0
Uppercase permutation 4	12/15	80.0	0
Uppercase permutation 5	10/15	66.7	0
Ordinary digits	12/15	80.0	1
Permuted digits	10/15	66.7	1
Ordinary number words	12/15	80.0	0
Conflicting number words	10/15	66.7	1
Neutral nonce labels	12/15	80.0	1

All 55 easy conditions were correct; medium and hard accuracy was 50/55 and 19/55. The six mappings over the identical uppercase symbol set ranged from 10/15 to 13/15, with the standard mapping at 11/15. Ordinary digits, ordinary number words, and neutral nonce labels each scored 12/15. Permuted digits and conflicting number words each scored 10/15.

Table 10: Paired binding outcomes. Discordance is first-condition-only/second-condition-only success.

Comparison	First	Second	Discordant
Digits	12/15	10/15	2/0
Number words	12/15	10/15	3/1
Nonce/conflicting words	12/15	10/15	2/0

For ordinary versus permuted digits, two puzzles were solved only under the ordinary mapping and none only under the permutation. Ordinary versus conflicting number words had three ordinary-only and one conflict-only success; neutral nonce labels versus conflicting words had two neutral-only and no conflict-only success. These directional differences are small relative to the 15-puzzle sample and therefore provide descriptive, not conclusive, evidence of semantic interference. Equal truncation counts in the digit and nonce/conflicting comparisons show that their two-answer gaps were not produced by differential truncation.

4.8 Prompt and output ablations

The exploratory ablation experiment recorded all 171 planned conditions and its completion marker before Slurm terminated backend teardown at the 15-hour wall-time. The request records contain 0 operational failures. Overall accuracy was 103/171 (60.2%), with 12 truncated responses. Table 11 reports every variant.

Table 11: Exploratory prompt and output ablations ($n = 9$ per variant).

Factor	Variant	Correct	Accuracy (%)
Rules	Minimal	5/9	55.6
Rules	Explicit constraints	6/9	66.7
Rules	Constraints and alphabet	5/9	55.6
Rules	Fully explicit	5/9	55.6
Mapping	Alphabet only	6/9	66.7
Mapping	To digits	8/9	88.9
Mapping	To abstract values	5/9	55.6
Output	Spaced rows	5/9	55.6
Output	Compact rows	3/9	33.3
Output	81-symbol string	3/9	33.3
Output	JSON	5/9	55.6
Empty marker	Dot	7/9	77.8
Empty marker	Zero	6/9	66.7
Empty marker	Underscore	4/9	44.4
Empty marker	EMPTY	6/9	66.7
Latin case	Uppercase	5/9	55.6
Latin case	Lowercase	7/9	77.8
Nonce case	Uppercase	8/9	88.9
Nonce case	Lowercase	4/9	44.4

Rule wording ranged only from 5/9 to 6/9 and showed no advantage for the fully explicit prompt. Declaring a digit mapping scored 8/9, compared with 6/9 for alphabet-only and 5/9 for abstract-value mapping. Neither compact rows nor an 81-symbol string improved on spaced rows or JSON. Empty-marker results ranged from 4/9 for underscore to 7/9 for dot. Case effects reversed across families: lowercase Latin exceeded uppercase Latin (7/9 versus 5/9), whereas uppercase nonce labels exceeded lowercase nonce labels (8/9 versus 4/9).

Four named variants—fully explicit rules, alphabet-only mapping, spaced output, and dot empty marker—produce identical prompts for each puzzle under the one-factor construction. Their independent sampled runs nevertheless scored 5/9, 6/9, 5/9, and 7/9. This two-answer spread among exact prompt replicates demonstrates substantial sampling variability at this scale. The larger ablation contrasts are useful diagnostics, but none warrants changing the frozen main protocol on nine puzzles alone.

5 Discussion

The complete main benchmark demonstrates that GPT-OSS-120B can solve Sudoku reliably under several symbol systems while still exhibiting representation-dependent failures. The model solved 95.0% of easy requests, showing that the generation and evaluation pipeline can produce exact grids rather than merely plausible completions. Across the same 60 puzzles, representation accuracy varied by 13.3 percentage points, from 61.7% for Greek letters to 75.0% for Arabic digits and uppercase Latin.

The full ordering cannot be summarized as a simple Unicode penalty. Bengali numerals reached 71.7%, equal to ASCII nonce labels and above emoji and abstract symbols at 63.3%. Uppercase Latin matched Arabic at 75.0%, whereas lowercase Latin reached 73.3%. These contrasts show that neither ASCII status, familiarity, nor visual complexity alone orders performance.

Difficulty effects were monotonic in the complete benchmark: easy accuracy exceeded medium accuracy, which exceeded hard accuracy. The 95.0%, 67.2%, and 44.4% tier results also show why exact counts by difficulty are necessary when interpreting representation averages. The paired retention

analysis further shows that equal marginal accuracy need not mean invariant behaviour: uppercase Latin and Arabic both scored 45/60, yet only 36 of the 45 Arabic successes were retained under uppercase Latin, with nine successes changing in each direction.

Long hidden analyses imposed substantial computational cost. Main-benchmark responses averaged 290.8 seconds and 54756 completion tokens. Twenty-four responses exhausted the 131,072-token ceiling. These were model-output failures rather than infrastructure failures: the backend completed every recorded main and input/output-cross request, and neither experiment had an operational error. The distinction matters because increasing scheduler wall-time permits long requests to finish, whereas increasing an exhausted generation ceiling would change the inference protocol.

The completed input/output cross narrows but does not fully identify the mechanism. Perfect performance on all 75 no-Sudoku controls shows that short copying, lookup, counting, mapping translation, and full-grid conversion were not independently difficult under these prompts. Equal 12/15 accuracy for the Arabic-to-Arabic and Greek-to-Greek Sudoku baselines gives no evidence of a Greek-output penalty in this 15-puzzle sample. The lower cross-mapping results, 10/15 and 11/15, instead suggest that requiring an explicit input-output remapping can add difficulty. Because these differences involve only a few discordant puzzles and are concentrated in harder Sudoku instances, they should not be interpreted as isolating a single causal stage.

The controlled token-length result likewise argues against a single-factor explanation. If token count alone caused a graded penalty, accuracy would be expected to decline from one- to two- to three-token labels. Instead, the two-token condition was highest. This does not show that tokenization is irrelevant: the sample is small, difficulty dominates many failures, and the planned regression was unidentifiable because the constructed features were collinear. It does show that the observed main-family ordering cannot be explained by a simple monotonic label-token count.

The binding results point in the expected direction for semantic interference but remain underpowered. Both familiar-label conflicts reduced accuracy from 12/15 to 10/15, and no puzzle was uniquely rescued by the digit permutation. At the same time, the 10/15–13/15 range across mappings of the same uppercase tokens shows that a fixed alphabet does not ensure identical item-level behaviour. Perfect easy accuracy and sharply lower hard accuracy again indicate that binding effects become most visible near the model’s reasoning limit.

The exploratory ablations do not identify a universal prompt or formatting remedy. Explicit digit mapping reached 8/9, but exact default-prompt replicates themselves varied from 5/9 to 7/9. Compact output did not improve accuracy, a more explicit rule prompt did not outperform minimal wording, and case effects reversed between Latin and nonce labels. These results caution against attributing a small condition difference to one wording choice without replication. The self-revision experiment provides the remaining planned mechanism test.

6 Conclusion

This work implements an exactly scored test of representation invariance in LLM Sudoku solving. GPT-OSS-120B passed all qualification requests, solved 47/60 pilot requests, and solved 372/540 requests in the complete frozen main benchmark without operational failures. Main accuracy differed across bijective representations of the same puzzle set, ranging from 61.7% to 75.0%, and declined from 95.0% on easy puzzles to 44.4% on hard puzzles. Among the 45 puzzles solved in Arabic, transformed-representation retention ranged from 68.9% to 82.2%. Perfect basic symbol-control performance, non-monotonic token-length results, mapping-sensitive binding outcomes, and the absence of a universal prompt-format remedy rule out several simple explanations. Familiar-label conflicts produced a small directional penalty, but the binding and ablation samples do not support a single isolated mechanism. Successful Sudoku solving by GPT-OSS-120B is therefore not fully invariant to value renaming, and the observed effect reflects an interaction among representation, puzzle difficulty, stochastic generation, and termination behaviour.

References

- [1] Kaj Bostrom and Greg Durrett. Byte pair encoding is suboptimal for language model pretraining. In *Findings of EMNLP*, pages 4617–4624, 2020. doi: 10.18653/v1/2020.findings-emnlp.414.
- [2] Iman Mirzadeh et al. GSM-Symbolic: Understanding the limitations of mathematical reasoning in large language models. In *International Conference on Learning Representations*, 2025.
- [3] OpenAI. GPT-OSS-120B model card. <https://huggingface.co/openai/gpt-oss-120b>, 2026. Pinned revision b5c939de8f754692c1647ca79fbf85e8c1e70f8a.
- [4] Radek Pelánek. Difficulty rating of sudoku puzzles: An overview and evaluation. *arXiv preprint arXiv:1403.7373*, 2014. doi: 10.48550/arXiv.1403.7373.
- [5] Phillip Rust, Jonas Pfeiffer, Ivan Vulić, Sebastian Ruder, and Iryna Gurevych. How good is your tokenizer? on the monolingual performance of multilingual language models. In *Proceedings of ACL-IJCNLP*, pages 3118–3135, 2021. doi: 10.18653/v1/2021.acl-long.243.
- [6] Jessica Seely, Yuki Imajuku, Tianyi Zhao, Edoardo Cetin, and Llion Jones. Sudoku-bench: Evaluating creative reasoning with sudoku variants. *arXiv preprint arXiv:2505.16135*, 2025. doi: 10.48550/arXiv.2505.16135.
- [7] Kulin Shah, Nishanth Dikkala, Xin Wang, and Rina Panigrahy. Causal language modeling can elicit search and reasoning capabilities on logic puzzles. In *Advances in Neural Information Processing Systems*, volume 37, 2024.
- [8] Kaya Stechly, Matthew Marquez, and Subbarao Kambhampati. GPT-4 doesn’t know it’s wrong: An analysis of iterative prompting for reasoning problems. *arXiv preprint arXiv:2310.12397*, 2023. doi: 10.48550/arXiv.2310.12397.
- [9] Jerry Wei et al. Symbol tuning improves in-context learning in language models. In *Proceedings of EMNLP*, pages 968–979, 2023. doi: 10.18653/v1/2023.emnlp-main.61.
- [10] Yibo Yan, Yujie Lu, Ruochen Xu, and Zhenzhong Lan. Do PhD-level LLMs truly grasp elementary addition? probing rule learning vs. memorization in large language models. *arXiv preprint arXiv:2504.05262*, 2025. doi: 10.48550/arXiv.2504.05262.